Flight-envelope haptic controller

ENGE X200 · Assignment 2 · STM32C031C6

Student name: [to be supplied]
Student ID: [to be supplied]

1,732 body words · AI-assisted development draft

Evidence status: analytical predictions and software validation only. Wokwi recording and observed emulator results remain to be supplied.

1. Purpose, interpretation and evidence

This project implements the flight-envelope haptic controller specified in Assignment 2 for ENGE X200 Microcontroller Programming. Three positional servos stand in for torque-producing actuators: pitch and roll indicate individual axis limits, while a separate tilt servo gives an earlier combined warning. The controller also manages manual and autopilot indications. The implementation separates a reusable control module from STM32 hardware access, so mathematical and state-transition behavior can be checked independently of peripheral configuration.

The assignment PDF is the controlling technical specification. Its pitch and roll equations differ from the earlier System Overview document, and its five-degree deadband applies when disengaging feedback rather than delaying entry. Those differences materially affect behavior. This report therefore uses the assignment equations and nominal entry limits. The brief leaves exact equality at some boundaries unspecified; the implementation assigns 30° and 50° to compliant and clamp entry respectively, and 20° to tilt compliant entry, avoiding undefined boundary cases.

This is an AI-assisted development draft, not a student-authored submission. The assignment states that AI code generation is prohibited and only editing assistance is allowed. Eligibility must therefore be resolved under the course rules. The report distinguishes analytical predictions and software checks from emulator observations. No Wokwi screen recording or observed emulator results are claimed; the required results video and matching observed results remain outstanding.

2. Orientation calculation and program structure

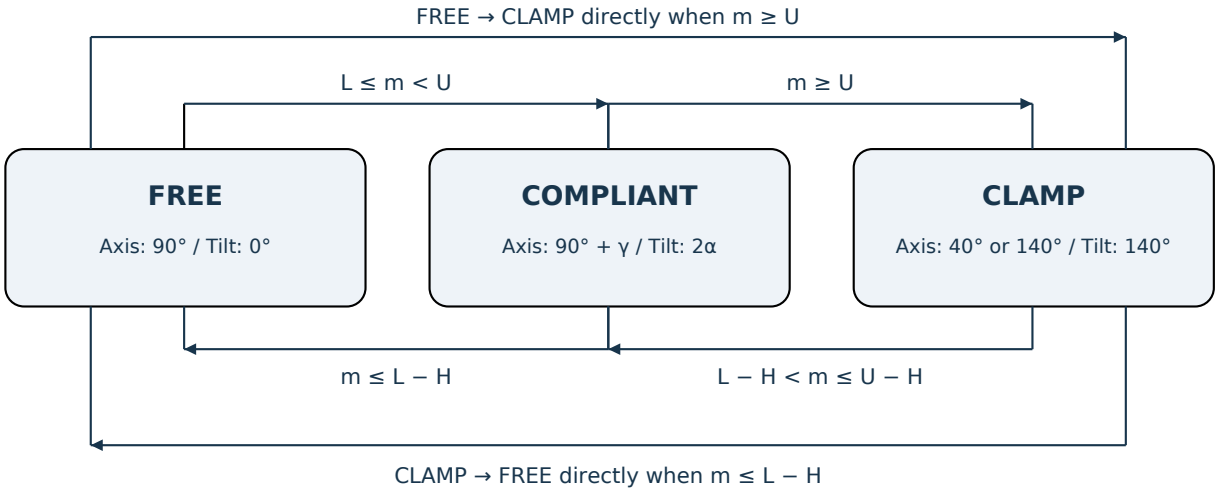
The angles_from_accel subroutine accepts signed 16-bit X, Y and Z accelerometer samples. It converts each value to g by dividing by 16384, matching the MPU6050 ±2 g setting. It computes roll as atan2(ay, az), pitch as atan2(−ax, √(ay² + az²)), and combined tilt as √(pitch² + roll²). Both atan2 outputs are multiplied by 180/π before applying degree-based thresholds. Using atan2 preserves signs and quadrants, including roll angles beyond 90°; replacing it with a simple ratio would lose this information.

A zero acceleration vector is rejected because it has no defined gravity direction. Non-finite or out-of-domain angles also trigger the neutral fault response. The calculations assume gravity-dominated, constant-velocity flight as stipulated; they do not distinguish gravity from sustained linear acceleration. No gyro fusion is claimed. Normalizing by the common sensor scale does not change the angles, but makes the units and test inputs explicit. The core avoids dynamic allocation and retains only the three region states, mode, fault flag and servo commands.

Each control update reads one accelerometer sample, calculates all three angles, updates the feedback states, evaluates the mode transition, and produces all servo commands from that same sample. Independent state storage is essential: a pitch transition must not overwrite roll history. The main loop requests a new sample every 20 ms, matching the 50 Hz servo command update rate. Button sampling and debouncing run more frequently so a held button produces one event rather than one toggle per control update.

3. Reusable feedback state machine

Figure 1 defines a reusable three-state machine with magnitude m, free boundary L, clamp boundary U, and hysteresis H = 5°. Pitch and roll use m = |angle|, L = 30° and U = 50°. Tilt uses its non-negative value, L = 20° and U = 70°. The same region_update function is called independently for each channel. Entry into stronger feedback occurs at the nominal limit; release requires crossing five degrees farther toward the center.



All unspecified conditions retain state. Clamp output sign follows current angle.

Figure 1. Reused region FSM. Axis parameters: L = 30°, U = 50°; tilt parameters: L = 20°, U = 70°. H = 5° in both cases.

From FREE, an input at or above U enters CLAMP directly; otherwise an input at or above L enters COMPLIANT. From COMPLIANT, m ≥ U enters CLAMP and m ≤ L−H returns to FREE. From CLAMP, m ≤ L−H returns directly to FREE; otherwise m ≤ U−H enters COMPLIANT. All other samples retain the current state. Direct transitions handle large slider steps without an artificial intermediate frame. Equality at each release threshold permits release, so a clamped pitch returns to compliant at 45° and compliant returns to free at 25°.

State retention explains why a displayed angle alone does not always identify the active region. For example, 28° is FREE when approached from the center but remains COMPLIANT when approached from 40°. Similarly, 47° remains CLAMP after a 60° input. This history dependence is intentional and suppresses chatter near the entry boundaries. Signed pitch and roll share magnitude-based region selection; their current sign determines which mechanical bound is commanded, including a large step directly from positive to negative clamp.

In FREE, pitch and roll commands are 90° and tilt is 0°. COMPLIANT pitch and roll commands equal 90° plus the signed detected angle; CLAMP selects 140° for positive input and 40° for negative input. Tilt COMPLIANT commands twice its detected angle, with a maximum of 140°, and tilt CLAMP holds 140°. Explicit saturation protects the actuator range. Both operating modes retain this same feedback law because the brief provides no separate autonomous flight-control

algorithm.

4. Autopilot, button events and fault handling

Figure 2 shows the mode state machine. Startup selects AUTOPILOT. In MANUAL, either a debounced button press or simultaneous pitch and roll CLAMP states causes AUTOPILOT. In AUTOPILOT, a button press permits MANUAL only when both region states are FREE and both raw angle magnitudes are strictly below 30°. Every other input leaves the mode unchanged. Tilt alone never changes the mode. The LEDs are mutually exclusive: PA2 indicates autopilot and PB11 indicates manual.

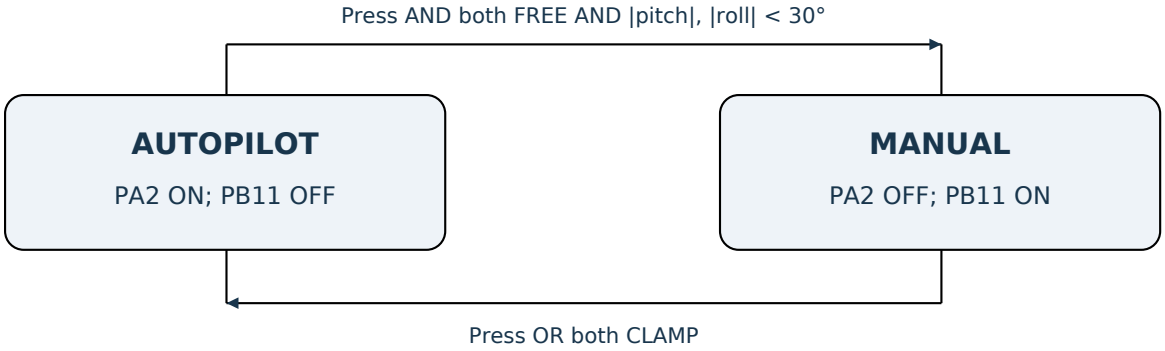


Figure 2. Mode FSM and LED state actions. A “press” means one debounced rising-edge event.

The manual handover checks state and present angle rather than trusting either alone. A state history can retain compliance even after the measured angle has fallen below 30°, while a raw angle at the free limit is not inside the strict free interval. Unsafe presses are consumed immediately rather than queued until the stick later centers. The button debouncer requires an unchanged candidate level for 30 ms, generates an event only on a stable rising edge, and requires release before another press. Unsigned elapsed-time arithmetic tolerates millisecond counter wraparound.

A failed sensor transaction, wrong device identity or invalid acceleration prevents stale data from controlling the outputs. The fault response returns pitch and roll to 90°, tilt to 0°, and the mode to AUTOPILOT. After a transaction timeout or NACK, the peripheral is reset and initialization is retried once per second. A persistently stuck physical bus still requires wiring or power repair. A later valid sample restores feedback from reset region history, while manual mode still requires a fresh safe button event. This deterministic response is a software fallback for the educational emulator; it is not a certified aircraft safety strategy.

5. Peripheral configuration and timing

The hardware layer uses STM32C031 CMSIS register definitions, with startup code initializing memory and the interrupt vector base, and a linker layout for 32 KiB flash and 12 KiB SRAM. The internal high-speed oscillator supplies a 48 MHz system and timer clock. TIM3 prescaler 47 produces a 1 MHz counter, and auto-reload 19999 produces a 20 ms period. PWM mode 1 with compare preload supplies roll on channel 1/PB4, pitch on channel 2/PB5, and tilt on channel 3/PB0. These timer pins use alternate function 1, correcting the earlier overview’s AF0 assertion.

The servo conversion maps 0–180° to 1000–2000 µs, so neutral pitch and roll use 1500 µs and neutral tilt uses 1000 µs. PWM channels are enabled only after neutral compare values have been prepared. One-microsecond timer ticks make the compare register value equal to pulse

duration in microseconds. Position-controlled emulator servos visualize limits but cannot reproduce physical force or torque; real hardware would additionally require calibrated linkage, suitable actuator power and mechanical protection.

I2C1 uses PB8/SCL and PB9/SDA as open-drain alternate function 6 outputs with external pull-ups. TIMINGR = 0x00B4274F targets Fast-mode operation below 400 kHz with the analog filter enabled; actual hardware rise/fall timing requires waveform verification. The MPU6050 does not support the MCU's possible 1 MHz Fast-mode Plus rate. Initialization checks WHO_AM_I, wakes the sensor, configures ±2 g acceleration, and reads registers 0x3B–0x40 together. PA12 is an active-high input with pull-down. Diagnostic USART1 output on otherwise unused PA9 reports startup, sensor initialization, button events and periodic status in the Wokwi Terminal. The vector base is explicitly set to flash before enabling SysTick, following ST's SystemInit; this corrects a startup omission found during investigation of the reported red-light-only symptom. Bounded transaction waits ensure a missing sensor cannot permanently freeze the control loop.

6. Methodical results and recording plan

Table 1 specifies gravity-consistent test vectors calculated from selected pitch θ and roll φ: ax = −sin θ, ay = cos θ sin φ, and az = cos θ cos φ. This construction gives a vector magnitude of one g. Values are rounded for entry into the emulator, with a small angle tolerance expected from integer conversion and slider resolution. Every independent case should begin from a centered input unless a previous state is explicitly identified.

Case / initial state	ax, ay, az (g)	θ / φ / α (°)	Pitch / roll / tilt (°)
Centered / reset	-0.00000, 0.00000, 1.00000	0 / 0 / 0.00	90 / 90 / 0.00
Pitch compliant / reset	-0.64279, 0.00000, 0.76604	40 / 0 / 40.00	130 / 90 / 80.00
Roll compliant / reset	-0.00000, 0.64279, 0.76604	0 / 40 / 40.00	90 / 130 / 80.00
Tilt only / reset	-0.30902, 0.29389, 0.90451	18 / 18 / 25.46	90 / 90 / 50.91
Pitch negative / reset	0.86603, 0.00000, 0.50000	-60 / 0 / 60.00	40 / 90 / 120.00
Roll negative / reset	-0.00000, -0.86603, 0.50000	0 / -60 / 60.00	90 / 40 / 120.00
Both clamp / MANUAL	-0.86603, 0.43301, 0.25000	60 / 60 / 84.85	140 / 140 / 140.00

Table 1. Analytically predicted vectors and servo commands, not observed Wokwi measurements. All vectors have unit magnitude before rounding. "Reset" means FREE region history.

The 40° pitch-only and roll-only orientations exercise individual compliant feedback while the other axis stays FREE. Their tilt angle is also 40°, so the tilt servo correctly moves to 80°; "only" refers to the isolated orientation axis, not the absence of tilt feedback. With pitch and roll both 18°, each remains FREE while combined tilt is about 25.46°, giving approximately 50.91° on the tilt servo. This supplies the required tilt-only feedback case without violating the individual free regions.

A simultaneous 60° pitch and 60° roll is feasible under the assignment's equations and produces tilt approximately 84.85°. Both individual servos clamp at 140°, the tilt servo clamps at 140°, and MANUAL must change to AUTOPILOT. Negative single-axis 60° cases verify the 40° opposite bound. For release evidence, sweep each axis from 60° through 47°, 45°, 28° and 25°; the expected states are CLAMP, CLAMP, COMPLIANT, COMPLIANT and FREE. Sweep tilt similarly through 75°, 67°, 65°, 18° and 15°.

The final Wokwi recording must show these state sequences, safe and rejected button transitions, LED changes, and servo positions while the accelerometer values remain visible. The

clean cross-build passed using GNU Arm GCC 14.2.1, C11 and strict warning checks. It produced ELF, BIN and HEX outputs using 10,492 of 32,768 flash bytes and 132 of 12,288 static RAM bytes; stack usage was not measured. All 7,372 host checks passed, including 253 unit-gravity vectors across roll quadrants, boundary and hysteresis transitions, servo limits, mode guards, fault recovery, debounce counter wraparound and PWM conversion. AddressSanitizer and UndefinedBehaviorSanitizer checks also passed; leak detection was disabled under the traced runner, and the core allocates no heap memory. Circuit JSON pin names were validated against the official Wokwi board definition. These checks establish software consistency and build compatibility, but not successful emulator execution. Recorded observations should be added alongside Table 1 only after this demonstration is actually performed. No numerical emulator measurements or completed video are fabricated in this draft.

References and attribution

[1] Adelaide University, ENGE X200 Microcontroller Programming, Assignment 2, supplied assignment.pdf, pp. 1–5, due 3 November 2026.

[2] STMicroelectronics, STM32C031x4/x6 datasheet and RM0490 STM32C0x1 reference manual. Peripheral definitions used from cmsis-device-c0, commit bcc8d94fecb767d1afb53d8c12a8f87ebeb503a2, github.com/STMicroelectronics/cmsis-device-c0.

[3] Arm, CMSIS Core headers, CMSIS_5 commit 55b19837f5703e418ca37894d5745b1dc05e4c91, github.com/ARM-software/CMSIS_5. Vendor files retain their licences in the code archive.

[4] InvenSense, MPU-6000 and MPU-6050 Product Specification, revision 3.4, and Register Map and Descriptions, revision 4.2. Relevant registers: PWR_MGMT_1, ACCEL_CONFIG, ACCEL_XOUT_H and WHO_AM_I.

[5] Wokwi documentation, STM32 Nucleo64 C031C6 and VS Code Getting Started, docs.wokwi.com. The project configuration targets board-st-nucleo-c031c6. Pin-specific alternate functions were cross-checked against STMicroelectronics/STM32_open_pin_data, commit 7d1f1514ed5583ec5007ad91236b4e1d377295b1.

Attribution: code and this draft report were generated with AI assistance. No lecture or practical source files were supplied or copied. Vendor CMSIS headers are third-party code, not original student work.